Safe-Stride Assist: An AI-Based Safe Walking Assistance System for People with Visual Impairments

Jhenkar P
Dept. of Computer Science and Engineering
The National Institute of Engineering
Mysuru, India
2023cs_JHENKARP_b@nie.ac.in

[Guide name to be added]
Dept. of Computer Science and Engineering
The National Institute of Engineering
Mysuru, India
[guide email to be added]

***Abstract*—Visually impaired pedestrians need to know whether an obstacle blocks their next steps, how far away it is, and what to do about it, but most camera-based assistants stop at drawing bounding boxes. This paper presents Safe-Stride Assist, a research prototype that turns a single camera stream into short, prioritised spoken guidance. The system combines YOLO11s detection, ByteTrack tracking, Depth Anything V2 monocular depth, and a geometric reasoning layer: RANSAC ground-plane fitting converts relative depth to approximate metres using the camera height, and a 1 m × 3 m walking corridor exposes known and unknown blockers and suggests a clearer side. Looming-based time-to-collision, persistence checks and cooldowns control when to speak, and a GPS module provides route instructions with compass alignment. In controlled scenes, the scale correction kept the distance error at 2.4 cm even when the raw depth was twice too large, while an uncorrected pipeline missed 50 % of the blockers at a ×1.5 scale error and all of them at ×3. A six-class hazard detector reached 0.556 mAP50 on 540 held-out images, and the full pipeline ran at 20.4 FPS on a laptop GPU (RTX 4060). Safe-Stride is an educational prototype, not a certified mobility device.**

Keywords*—*Safe walking assistance, visual impairment, monocular depth estimation, ground-plane estimation, obstacle detection, YOLO, ByteTrack, time-to-collision, route guidance

I. INTRODUCTION

People with visual impairment combine a white cane, a guide dog, memory of the environment and help from other people to move safely. A camera cannot replace these aids, but it can add a perception channel for things a cane reaches late or not at all: head-level objects, approaching vehicles, steps and unfamiliar barriers. The World Health Organization stresses that such support must be accessible and understandable [6].

Detection alone is not enough. A person 15 m away at the roadside is irrelevant, whereas an unlabelled bin 2 m ahead in the walking line is urgent. Safe-Stride Assist therefore treats four questions as first-class outputs: is the object in the user's path, how far is it, is it approaching, and what should the user do. The answer is delivered as a short sentence such as “Stop! Path blocked, 2 meters. Keep Left.”

The main contributions are: (i) a distance-aware walking-corridor method that uses depth points above an estimated ground plane, so unknown blockers are found without a trained class; (ii) a ranking and announcement policy that limits repetition; (iii) a route-guidance layer with heading alignment, distance countdown and off-route detection; and (iv) an implementation with a browser console and a phone client, together with an evaluation that separates controlled, simulated and real-video evidence.

II. PROBLEM STATEMENT

Pedestrians who cannot see well meet people, vehicles, poles, doorways, steps, uneven ground and temporary barriers. Conventional detectors recognise only trained classes and do not state whether an object blocks the path, how far it is, or whether it is approaching. Monocular depth networks give scene structure but not guaranteed metric scale [4]. The goal is a supplementary assistive prototype that (a) estimates approximate distance from one camera, (b) decides path relevance inside a bounded corridor, (c) detects blockers that have no class label, (d) ranks hazards by urgency and approach risk, (e) speaks sparingly, and (f) combines this with route guidance, while stating its limits honestly.

III. LITERATURE SURVEY

Redmon et al. [1] showed that one network pass can detect objects in real time, and current Ultralytics checkpoints [5] continue this line. Zhang et al. [2] proposed ByteTrack, which keeps identities by associating both high- and low-confidence boxes; temporal identity supports persistence and approach analysis. Yang et al. [3] introduced Depth Anything V2 for dense monocular depth, and Ranftl et al. [4] showed that dense prediction transformers capture scene structure, although a relative depth map is not a guaranteed distance.

Khan et al. [7] reviewed navigation assistants for blind and visually impaired people and Xu et al. [8] reviewed wearable obstacle-avoidance aids; both conclude that electronic aids should complement, not replace, traditional mobility tools and must consider sensing range and feedback. Katzschmann et al. [9] demonstrated local navigation with time-of-flight sensing and haptic feedback. Eltaher et al. [10] surveyed urban navigation systems, which motivates route guidance, heading alignment and off-route handling. Johannesson et al. [11] explored converting scenes to sound, and Shull and Damian [12] reviewed haptic wearables; both support multimodal, non-visual feedback. Ranganeni et al. [13] studied levels of control in a navigation assistant and found that preferences vary with the situation, and Zhang et al. [14] built a smartphone assistive navigation system on ARCore, which supports a portable phone-based design.

Drawbacks of existing systems

- Detection labels and boxes do not say whether an object lies in the walking path.
- Relative monocular depth is not metric; without calibration its distances can be wrong by a constant factor [4].
- Objects outside the trained classes (temporary barriers, poles, bins) may be missed entirely.
- Frame-by-frame systems cannot tell whether an object is approaching.
- Frequent announcements raise cognitive load and hide urgent warnings.
- Obstacle sensing and route guidance are usually separate systems.

Advantages of the proposed system

- Metric scale is recovered from the ground plane and a known camera height.
- Known objects and depth-based unknown blockers share one corridor decision.
- Persistence, cooldowns and priority ranking limit repeated speech.
- Everything runs locally: camera frames are not uploaded to a cloud service.
- The same engine serves a browser console and a phone client with GPS guidance.

IV. SYSTEM DESIGN

A. System architecture

Fig. 1 shows the pipeline. Frames come from a webcam, an uploaded video or the phone camera. The perception stage runs YOLO11s with ByteTrack and Depth Anything V2. The reasoning stage estimates the ground plane, corrects the depth scale and evaluates the corridor. The decision stage ranks alerts and decides when to speak. The output stage draws the annotated feed and speaks through Windows speech, the browser or the phone. Learned perception is kept apart from deterministic reasoning so each can be tested on its own.

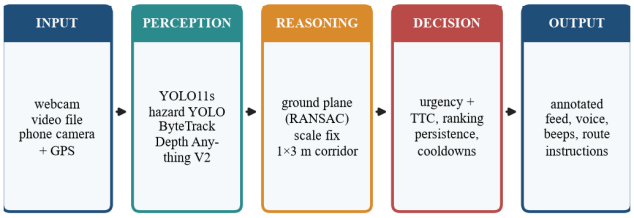


Fig. 1. Local perception-to-guidance pipeline of Safe-Stride Assist.

B. Walking-corridor geometry

The corridor is a 1 m wide and 3 m long region on the ground in front of the camera (Fig. 2). A point belongs to a blocker when it lies inside the corridor and between 0.25 m and 2.0 m above the ground, so curbs are ignored but head-level objects count. Objects outside the corridor, such as the grey box in Fig. 2, do not trigger path warnings.

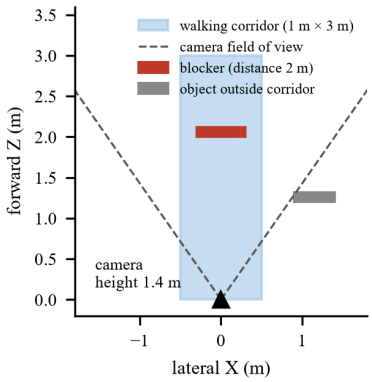


Fig. 2. Walking corridor in camera coordinates. A blocker inside it triggers a path warning.

C. Navigation state machine

Route guidance follows the states in Fig. 3. The route comes from an OpenStreetMap-based foot-routing service [16]. The phone reports GPS position and a compass-derived camera heading. The assistant first asks the user to face the road, then counts down the distance to the next maneuver, switches to the next instruction when the maneuver point is reached, and latches the arrived state.

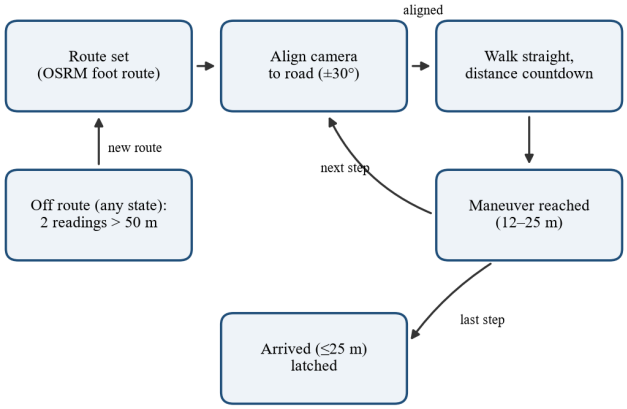


Fig. 3. State diagram of the route-guidance module.

E. Safety and privacy design

Camera frames are analysed on the user's own computer; the phone sends frames only to that computer over the local network, and no cloud vision service is used. The server keeps only the latest GPS position in memory and does not return it in its public status endpoints. Two parts do leave the machine: place search and routing send the destination text and the start and end coordinates to public OpenStreetMap-based services [16], and uploaded videos are stored in a local folder without automatic deletion. The server has no authentication, so it must run on a trusted network. Messages state approximate distances and the interface carries a prototype notice; the system never claims that a path is safe, only that no blocker was found.

V. SYSTEM IMPLEMENTATION

Table I lists the software and hardware environment in which all results were measured. The browser console and the phone client both talk to the same Python engine.

TABLE I. SOFTWARE AND HARDWARE ENVIRONMENT

Component	Specification
Operating system	Windows 11 (server-side voice uses Windows SAPI)
Language / runtime	Python 3.12 (3.10 or newer), PyTorch 2.5.1 with CUDA 12.4
Detection and tracking	Ultralytics 8.4 YOLO11s, ByteTrack
Depth	Depth Anything V2 Metric-Outdoor-Small (Transformers 4.49)
Image processing	OpenCV 4.10, NumPy 1.26
Back-end	Python HTTP server, MJPEG stream, JSON endpoints
Browser console	HTML, CSS, JavaScript, browser speech synthesis
Phone client	Expo SDK 58, React Native 0.88 (camera, GPS, compass, speech)
Hardware	Laptop with NVIDIA GeForce RTX 4060 Laptop GPU; phone with camera, GPS and compass

A. Phase 1: perception

YOLO11s runs at a 0.35 confidence threshold on walking-relevant COCO classes (people, vehicles, benches, chairs, hydrants and similar). A second YOLO11s model detects street hazards. It was trained from COCO weights on a South-Asian street dataset [15] that contains 22 classes (rickshaws, CNG vehicles, footpaths and others). Only six hazard classes that COCO does not know were kept (Table II); the tree class was dropped because it had two validation and no test examples, and polygon labels were converted to boxes. Training ran for 60 epochs (batch 32, 640 px, patience 15, default mosaic, colour and scale augmentation) in 0.67 h on a Tesla T4. Per-class thresholds follow the test results: the vague "obstacle" class needs 0.5 confidence, and "bin" is disabled because its recall was 0.026. ByteTrack gives each box a stable identity. Depth Anything V2 Metric-Outdoor-Small runs in FP16 at a 392-pixel input and is refreshed every second frame. The real-video runs in Section VII-D used a single-class stairs variant of the hazard detector (validation mAP50 0.50), because the six-class weights evaluated in Table IV were not in the project's model folder.

TABLE II. HAZARD DATASET AFTER FILTERING (INSTANCES PER SPLIT)

Class	Train	Valid	Test
stairs	183	19	14
pole	1637	107	92
pillar	510	193	175
bin	36	25	38
cone	123	32	74
obstacle	234	10	143
Images with hazards	1935	210	211
Background images	967	341	329

B. Phase 2: scale-corrected 3D geometry

Each depth value Z at pixel (u, v) is back-projected with a pinhole model, where f = (max(W, H)/2) / tan(θ/2) is the focal length in pixels and θ = 70° is the assumed field of view:

$$X = (u - W/2) \cdot Z / f,\ Y = (v - H/2) \cdot Z / f$$

The lower 40 % of the image is searched for the ground with RANSAC, choosing the plane $n \cdot P + d = 0$ that has most inliers within an adaptive tolerance $\tau = 0.03 \cdot \text{median}(Z)$; planes whose normal is not close to vertical are rejected:

$$\hat{\pi} = \text{argmax}_{\pi} \Sigma_{p \text{ in } S}\, 1(|n \cdot p + d| < \tau)$$

With the assumed camera height $H_c = 1.4$ m, the plane gives a scale factor per frame, and the median over the last 90 frames (about 3 s) removes occasional bad fits:

$$s_t = H_c / (-d_t),\ \hat{s}_t = \text{median}(s_{t-89}, \ldots, s_t)$$

$$h(P) = -\hat{s}_t (n \cdot P + d),\ \hat{P} = \hat{s}_t \cdot P$$

Points with h between 0.25 m and 2.0 m inside the corridor ($|X| < 0.5$ m, $0 < Z < 3$ m) form the blocker set. At least 15 points are required, the distance is the 5th percentile of their depth, the side with fewer points is suggested as the clear side, and an object with nothing below 1 m is flagged as head-level. A YOLO box on stairs is kept only when the depth slope inside the box confirms a direction (least-squares slope of height against depth):

$$dh/dZ > 0.08\text{: stairs up; } dh/dZ < -0.08\text{: stairs down}$$

C. Phase 3: risk, alert ranking and speech

Time-to-collision uses looming, which needs no depth: for a box of pixel height b(t), TTC = 1 / (d ln b / dt), with the slope fitted over the last second (at least five samples, boxes taller than 40 px, approach faster than 0.3 m/s, TTC below 10 s). Urgency combines proximity and time pressure and is scaled by a class weight w_c that reflects how harmful a collision would be (1.0 for vehicles and stairs, 0.7 for people, 0.6 for static objects):

$$U = w_c \cdot \max(1 - D/R,\ 1 - TTC/T)$$

Here D is the distance, R = 4 m (6 m for hazards) is the proximity range and T = 6 s the time range. An object in the red corridor is always at least a warning, and "Stop!" is spoken when $U \geq 0.6$ or the object is in the path. The announcer waits 0.2 s before speaking about an off-path object, repeats an alert only after 5 s and only if the object is 1 m closer, and forgets an alert after 3 s of absence so that a returning obstacle is announced again. Speech uses Windows SAPI with a stereo-panned beep from the obstacle side; the browser and the phone use their own speech engines.

D. Phase 4: route guidance and interfaces

Distances use the haversine formula and directions use the initial bearing θ:

$$d = 2 R_e \cdot \text{asin}(\sqrt{(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cos\varphi_2 \sin^2(\Delta\lambda/2)) })$$

The user is told to turn when the camera heading differs from the next road bearing by more than 30°. A step advances within 12–25 m of the next maneuver (scaled by GPS accuracy), an off-route warning needs two consecutive readings more than 50 m from the route, and arrival is declared within 25 m. The back-end is a Python HTTP server that streams the annotated frames to a browser console; the phone client (Expo / React Native) posts JPEG frames, GPS and compass headings, and speaks the returned alerts.

VI. TESTING

Table III lists scenarios with expected and obtained behaviour. Each row was produced by running the project code on a synthetic scene or a simulated route, not by hand. All 10 of 10 scenarios passed. In addition, an automated suite of 37 tests (pytest) covers navigation, spatial geometry, the announcer and the web server. During this verification the suite exposed four real defects that were fixed before the measurements below: inverted stairs up/down labels, a depth cache that never refreshed when depth was requested for every frame, muted re-announcement of a returning obstacle, and a distance countdown that stayed constant while walking.

TABLE III. SCENARIO TEST CASES (EXPECTED VS. OBTAINED)

ID	Scenario	Expected	Obtained	Status
TC1	Surface rising away from user (slope +0.5)	stairs up	stairs up	Pass
TC2	Surface falling away from user (slope -0.5)	stairs down	stairs down	Pass
TC3	Flat ground inside the stairs box	no direction	no direction	Pass
TC4	Obstacle 2.0 m ahead, depth scale ×2	blocked, 2.0 ± 0.3 m	blocked, 2.0 ± 0.3 m (1.95 m)	Pass
TC5	Obstacle 1.2 m to the side (outside corridor)	path clear	path clear	Pass
TC6	Obstacle slightly right of centre	keep left	keep left	Pass
TC7	Detection with undefined distance	no alert	no alert	Pass
TC8	Same obstacle returns after path cleared	announced again	announced again	Pass
TC9	GPS 150 m off the route (two readings)	off-route warning	off-route warning	Pass
TC10	Arrival, then GPS jitter 40 m away	arrival announced once	arrival announced once	Pass

VII. RESULT ANALYSIS

A. Hazard detector

On the held-out test split (540 images, 536 instances, executed notebook) the six-class model reached precision 0.667, recall 0.584, mAP50 0.556 and mAP50-95 0.401 at 11.4 ms per image on a Tesla T4 (Table IV, Fig. 4). The validation split gave a higher mAP50 of 0.684, so the model generalises less well to unseen images than validation suggests. Cones and pillars are detected well. Bins are almost never found (recall 0.026) because the training set contained only 36 bin examples, so the live system disables that class. The stairs estimate rests on only 14 test instances and should be read as indicative.

TABLE IV. HAZARD-MODEL TEST RESULTS PER CLASS

Class	Inst.	P	R	mAP50	mAP50-95
stairs	14	0.660	0.555	0.571	0.437
pole	92	0.570	0.739	0.585	0.418
pillar	175	0.728	0.680	0.746	0.589
bin	38	0.652	0.026	0.034	0.029
cone	74	0.972	0.944	0.942	0.707
obstacle	143	0.419	0.559	0.458	0.228
all	536	0.667	0.584	0.556	0.401

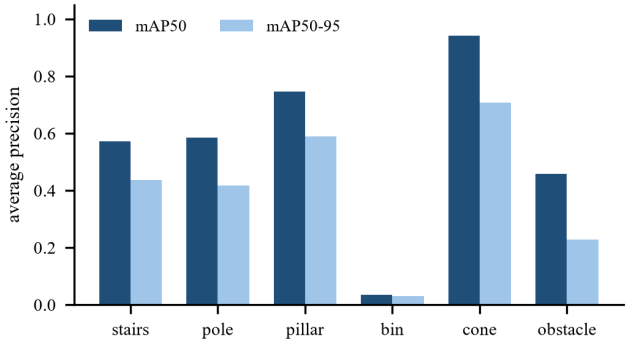


Fig. 4. Per-class average precision of the hazard detector on the test split.

B. Effect of ground-plane scale correction

A synthetic scene with exact geometry (flat ground, a 0.8 m wide and 1.2 m tall obstacle at 1.5–2.8 m, 2 % depth noise, five noise seeds) isolates the contribution of scale correction. The raw depth was multiplied by 1.0, 1.5, 2.0 and 3.0 to imitate a monocular network that is wrong by a constant factor. With correction the mean distance error was 2.4 cm at every factor and every blocker was detected. Without it, the error was 0.78 m at ×1.5 and 1.38 m at ×2, and only 50 % (×1.5) and 25 % (×2) of blockers were still inside the 3 m corridor; at ×3 none were (Fig. 5). These scenes are an idealised upper bound: real scenes have uneven ground, occlusion and a camera height that changes with the user.

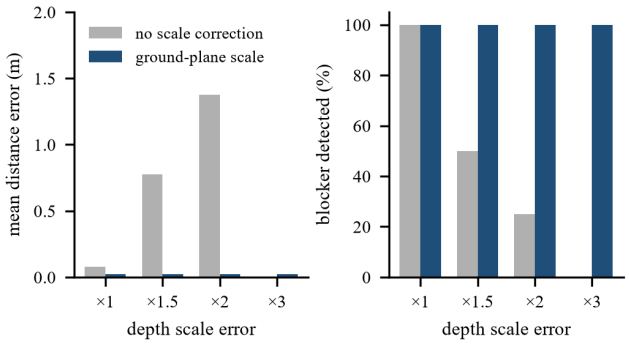


Fig. 5. Distance error (left) and blocker detection rate (right) with and without ground-plane scale correction.

C. Speed

Table V gives the per-frame time measured on an NVIDIA GeForce RTX 4060 Laptop GPU for a walking video (1280 px wide), with the COCO detector, the hazard detector, depth, spatial analysis and drawing. Refreshing depth on every second frame lowers the total from 62.2 ms (16.1 FPS) to 49.0 ms (20.4 FPS), and depth is the dominant cost. Risk analysis and the announcer take about 0.1 ms.

TABLE V. MEAN TIME PER FRAME (ms)

Stage	Depth every frame	Depth every 2nd frame
detect (COCO)	12.3	12.9
detect (hazards)	10.9	11.3
depth	29.7	15.1
spatial	7.2	7.5
risk + announcer	0.1	0.1
drawing	1.9	2.1
TOTAL	62.2	49.0

D. Behaviour on real walking video

The full pipeline was run on 5 clips (90 s in total): a street clip, a crowded station walkway, a city sidewalk with a construction barrier, an indoor lobby, and a short indoor phone recording of a living room. There is no hand-labelled ground truth, so Table VI reports what can be measured: speed (mean 14.6 FPS including all models), alert rate and the share of frames with a blocked path. Fig. 6 shows a typical annotated frame with its depth map. The living-room clip is a failure case: the stairs detector fires on the patterned carpet and the path is reported blocked in every frame, which produces 175 alerts per minute. It shows that the stairs model, trained on outdoor scenes, is not reliable on indoor floors, and that the alert rate on the other four clips (6–48 per minute) is more representative. Whether individual stairs alerts on the street clips were correct was not checked by hand. Messages spoken in these runs include “Truck approaching at 11 o'clock, 3 meters”, “Stop! Path blocked, 3 meters. Keep Left.”, “Person approaching at 1 o'clock, 6 meters” and “Stop! person ahead, 3 meters”; their correctness was likewise not verified. As a distance sanity check, the depth-based distance to fully visible people was compared with the distance implied by a 1.7 m person height: the ratio had a median of 1.08 (interquartile range 0.87–1.34, 12111 measurements). Both estimates rely on assumptions, so this tests consistency, not absolute accuracy.

TABLE VI. FULL-PIPELINE BEHAVIOUR ON WALKING VIDEO

Clip	Length (s)	FPS	Alerts per min	Urgent	Blocked (%)
street	20	16.6	6.0	0	0
living room*	10	21.8	175.3	28	100
walk_001-1	20	14.5	48.0	8	2
walk_002-1	20	9.2	24.0	6	0
walk_003-2	20	10.9	15.0	5	39



Fig. 6. Prototype output on the sidewalk clip: annotated frame (left) with distance, clock direction, time-to-collision (magenta) and path status, and the metric depth map (right). *Failure case, see text.

E. Time-to-collision accuracy

The looming estimator was tested on a simulated 1.7 m tall person approaching a 1280×720 camera at constant speed from 10 s before impact, with 1.5 px of noise on both box edges, 30 frames per second and 20 noise seeds per speed. Table VII lists the mean estimate at four true values of time-to-collision. An estimate was available at every checkpoint, so the 6 s urgency range of the alert score is covered. The estimate was too large by 0.35–0.52 s at every speed, close to half of the one-second window used for the slope fit: the fit reports the average over the last second, so it lags. The alert therefore arrives about half a second later than ideal, which is 1.5 m of travel for a 3 m/s bicycle. Subtracting half the window would likely remove most of this bias, but that was not tested.

TABLE VII. ESTIMATED TIME-TO-COLLISION (s) AT FOUR TRUE VALUES

Closing speed	True 6.0	True 4.0	True 3.0	True 2.0
1 m/s	6.42	4.50	3.48	2.48
1.5 m/s	6.36	4.50	3.48	2.48
3 m/s	6.35	4.52	3.48	2.49

F. Route guidance under GPS noise

A 155 m two-leg route was walked 20 times per noise level at 1.4 m/s with Gaussian GPS error, a compass error of 8° and a one-second sample interval (Table VIII). Navigation reached the turn and the destination in all runs, the distance countdown decreased monotonically between the first and last message in all runs, and no run raised a false off-route warning. About 8.7 messages per 100 m were spoken. These are simulations; the compass-to-heading conversion on a real phone has not been validated on a device.

TABLE VIII. ROUTE-GUIDANCE SIMULATION (20 RUNS PER ROW)

GPS σ (m)	Arrived (%)	Turn step (%)	False off-route (%)	Msgs/100 m
3	100	100	0	8.9
6	100	100	0	8.8
10	100	100	0	8.4

VIII. DISCUSSION AND LIMITATIONS

Three results shape the design. First, scale correction is what makes a 3 m corridor usable with a monocular network: without it the distance error grows in proportion to the scale error and blockers leave the corridor. Second, depth estimation is the largest cost (Table V), so refreshing it every second frame buys about four frames per second at the price of slightly older geometry. Third, the hazard detector generalises less well than its validation score suggests (0.684 versus 0.556 mAP50), and the classes with few examples (stairs, bins) are the weakest, so rare but dangerous hazards need more data. Verification also paid off: the explicit scenario tests caught four defects that no single screenshot would have shown.

Monocular depth and a fixed camera height make distances approximate; camera pitch, lighting, motion blur and reflective surfaces reduce accuracy, and a phone held at a different height changes the scale. The controlled experiments use idealised geometry. The real-video clips have no ground-truth labels, the stairs detector produced false alarms on an indoor patterned floor and on an ordinary sidewalk, the hazard test set has few stairs examples, the bin class is not usable, and the weights in the repository's live configuration are a single-class stairs model rather than the six-class model evaluated in Table IV. No study with visually impaired users has been carried out. The prototype has not been validated for safety and must be used only together with a cane, a guide dog or human support.

IX. CONCLUSION AND FUTURE WORK

Safe-Stride Assist shows that one camera can give path-aware, prioritised guidance when detection, tracking, scale-corrected monocular depth and corridor reasoning are combined, running at about 20 FPS on a laptop GPU. Ground-plane scale correction removed the large distance errors that a constant depth-scale error produces, and the alert policy and route module behaved consistently in simulation. Future work is to evaluate with labelled walking data and with users, calibrate risk weights, estimate camera height and pitch from the phone sensors, retrain the bin and obstacle classes with more data, add haptic feedback, and validate compass handling on devices.

REFERENCES

[1] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You only look once: Unified, real-time object detection," in Proc. IEEE CVPR, 2016.

[2] Y. Zhang, P. Sun, Y. Jiang et al., "ByteTrack: Multi-object tracking by associating every detection box," in Proc. ECCV, 2022.

[3] L. Yang, B. Kang, Z. Huang et al., "Depth Anything V2," in Advances in Neural Information Processing Systems, 2024.

[4] R. Ranftl, A. Bochkovskiy, and V. Koltun, "Vision transformers for dense prediction," in Proc. IEEE ICCV, 2021.

[5] Ultralytics, "Ultralytics YOLO documentation." [Online]. Available: https://docs.ultralytics.com/

[6] World Health Organization, World Report on Vision. Geneva: WHO, 2019.

[7] S. Khan, S. Nazir, and H. U. Khan, "Analysis of navigation assistants for blind and visually impaired people: A systematic review," IEEE Access, vol. 9, pp. 26712–26734, 2021.

[8] J. Xu et al., "Wearable obstacle avoidance electronic travel aids for blind and visually impaired individuals: A systematic review," IEEE Access, 2023.

[9] R. K. Katzschmann, B. Araki, and D. Rus, "Safe local navigation for visually impaired users with a time-of-flight and haptic feedback device," IEEE Trans. Neural Syst. Rehabil. Eng., vol. 26, no. 3, pp. 583–593, 2018.

[10] M. Eltaher et al., "A systematic review of urban navigation systems for visually impaired people," Sensors, vol. 21, no. 9, p. 3103, 2021.

[11] O. I. Johannesson et al., "The Sound of Vision project: The concept and first results," Brain Sci., vol. 6, no. 3, p. 20, 2016.

[12] P. B. Shull and D. D. Damian, "Haptic wearables as sensory replacement, sensory augmentation and trainer," J. NeuroEng. Rehabil., vol. 12, p. 59, 2015.

[13] V. Ranganeni et al., "Exploring levels of control for a navigation assistant for blind travelers," in Proc. ACM/IEEE HRI, 2023, pp. 4–12.

[14] C. Zhang et al., "An ARCore based user centric assistive navigation system for visually impaired people," Appl. Sci., vol. 9, no. 5, p. 989, 2019.

[15] Mendeley Data, "Object detection dataset: Navigation assistance for the visually impaired people using YOLOv11," dataset ID m68g3h7p87. [Online]. Available: https://data.mendeley.com/

[16] OpenStreetMap contributors, "OpenStreetMap." [Online]. Available: https://www.openstreetmap.org/